Earned Autonomy Wallet + Financial Advisor

> বাংলাদেশের জন্য একটি **সিমুলেটেড পেমেন্ট ওয়ালেট**: AI বিল ও লেনদেন প্রস্তুত করতে পারে; প্রতিটি চূড়ান্ত সম্পাদনে ব্যবহারকারীর নতুন ডিভাইস যাচাইকরণ প্রয়োজন।

AI কী করে

খরচ বুঝে সাশ্রয়ের পরামর্শ দেয়, বিল চিনে সময়সূচি বানায়, বিশ্বস্ততা অর্জনের পর প্রস্তাব ও পেমেন্টের খসড়া নিজে তৈরি করে।

AI কী পারে না

নিজে টাকা সরাতে পারে না। Guardian, ব্যবহারকারীর অনুমোদন ও সার্ভারের লেজার ছাড়া কোনো পেমেন্ট হয় না।

এই সংশোধনের মূল নিয়ম

বিশ্বাস অর্জন = **প্রস্তুতি ও সময়সূচিতে** বেশি স্বায়ত্তশাসন। বিশ্বাস অর্জন কখনোই চূড়ান্ত WebAuthn অনুমোদন বাদ দেয় না।

> **ডেমো সীমা:** এখানে ৳ ব্যালান্স ও বিল পরিশোধ শুধু স্থানীয় সিমুলেশনে হবে। ব্যাংক, bKash, Nagad বা অন্য PSP-তে প্রকৃত টাকা পাঠানো হবে না।

মূল ১০-পৃষ্ঠার গাইডের Advisor, বিল-ব্যবস্থাপনা, Wilson lower bound, Guardian, audit, শুরু থেকে বানানোর ধাপ এবং ন্যূনতম ২০টি পরীক্ষা বজায় রেখে এই সংস্করণ তৈরি। ১৮ অক্টোবর ২০২৬-এ যাচাই করা তথ্যের ভিত্তিতে প্রতিযোগিতার অবস্থান লেখা হয়েছে।

১ · বিষয় নির্বাচন ও সত্য দাবি

বিষয়	এই প্রকল্পের অবস্থান
Topic F Agent Wallet / Autonomous On-chain Wallet	প্রকাশিত বিবরণে এটি স্পষ্টত **on-chain** বিষয় [১]। এই নকশায় কোনো blockchain বা real on-chain transaction hash নেই। তাই মূল গাইডে উল্লিখিত on-chain hash শর্ত পূরণ হয়েছে বলে দাবি করা যাবে না। বিস্তারিত নিয়মে hash-এর ভাষা আলাদাভাবে যাচাই করতে হবে।
Topic C Cross-border & Student Finance Assistant	শুধু বাংলাদেশি বিল ও ওয়ালেট হলে উপযুক্ত নয়। প্রকৃত আন্তর্জাতিক শিক্ষার্থী/সীমান্তপার অর্থব্যবস্থার সমস্যা, ব্যবহারকারী গবেষণা ও বৈধ সমাধান থাকলে বিবেচ্য; নইলে আয়োজকের লিখিত ব্যাখ্যা চাইতে হবে [১]।
Topic A Personal Finance Assistant	খরচ বিশ্লেষণ, বিল ও সাশ্রয়ের পরামর্শের জন্য প্রকাশিত বর্ণনার সঙ্গে সবচেয়ে সরাসরি মেলে। তবে জমার ফর্মে কোন বিষয় গ্রহণযোগ্য তা আয়োজককে নিশ্চিত করা ভালো [১]।

জমার সময় ও যোগ্যতা

আন্তর্জাতিক ট্র্যাকের প্রকাশিত সময়সূচি অনুযায়ী **২০ অক্টোবর ২০২৬** নিবন্ধন ও জমার শেষ তারিখ, সময় অঞ্চল UTC+8; নির্দিষ্ট ঘড়ির সময় প্রকাশিত সারাংশে নেই। আগেই জমা দিন। ২–৫ সদস্য; সবাই চীনের বাইরে বিশ্ববিদ্যালয়ের পূর্ণকালীন শিক্ষার্থী; অন্তত ৬০% অ-চীনা নাগরিক [১]।

> **ভাষা:** আন্তর্জাতিক ট্র্যাকের দাপ্তরিক ভাষা ইংরেজি [১]। এই বাংলা গাইড দিয়ে কাজ তৈরি করুন; চূড়ান্ত ডেক/বর্ণনা/ডেমো ইংরেজিতে প্রস্তুত করুন।

[১] IEEE CIS Singapore, “2026 FinTechathon”, ieee-cis-singapore.org/2026-fintechathon (১৮ অক্টোবর ২০২৬ দেখা)।

২ · ওয়ালেট, অনুমোদন ও ভূমিকা

ওয়ালেট বলতে এখানে কী বোঝায়?

সিমুলেটেড ওয়ালেট একটি অ্যাপের হিসাবখাতা। ডেমোতে ৳৫,০০০ যোগ করলে *demo balance* বাড়ে; কোনো ব্যাংক হিসাবে আসল টাকা যোগ হয় না। প্রত্যেক সফল পেমেন্টে অভ্যন্তরীণ লেজারে সমান ডেবিট ও ক্রেডিট লেখা হয়। **ডেমো রসিদ ID** হলো অ্যাপের রসিদ, blockchain transaction hash নয়।

অংশ	কাজ	পেমেন্ট সম্পাদনের ক্ষমতা
Advisor / AI Agent	CSV ও বিল থেকে তথ্য, পরামর্শ, খসড়া	নেই
Scheduler	বিলের তারিখ দেখে কাজ প্রস্তুত ও সতর্কতা	নেই
Guardian	স্থির নিয়ম, সীমা, বিশ্বাস, ঝুঁকি যাচাই	শুধু অনুমোদনযোগ্য কি না বলে
WebAuthn verifier	এই নির্দিষ্ট পেমেন্টে নতুন user verification যাচাই	নিজে লেজার বদলায় না
Ledger executor	সব শর্ত পূরণ হলে একবারই ডেমো এন্ট্রি লেখে	শুধু সার্ভারে

পেমেন্টের যাত্রা

বিল / CSV → AI খসড়া → Guardian → Pending approval → বিস্তারিত দেখানো → ফোন/ল্যাপটপে WebAuthn → সার্ভার যাচাই → সিমুলেটেড লেজার + audit

সততা: "ফিঙ্গারপ্রিন্ট" UI-তে লক্ষ্য হতে পারে, কিন্তু সাধারণ WebAuthn সার্ভার জানে *user verification* হয়েছে; সেটি আঙুল, ডিভাইস PIN, বা অন্য স্থানীয় পদ্ধতি ছিল কি না সাধারণত জানে না [২]। এই গাইডে ডিভাইস PIN fallback গ্রহণযোগ্য। তাই "প্রতি পেমেন্টে verified passkey approval" দাবি করুন, "ফিঙ্গারপ্রিন্ট ছাড়া কখনো সম্ভব নয়" দাবি নয়।

৩ · সম্পূর্ণ আর্কিটেকচার

React / Vite ওয়েব UI
Dashboard · Advisor · Bills · Trust · Approvals · History

↓ HTTPS + session + CSRF protection

FastAPI সার্ভার
CSV parser · Agent adapter · Bill scheduler · Guardian · WebAuthn service

↓ শুধু যাচাইকৃত সার্ভার কোড

SQLite ডেমো ডেটাবেস
transactions · bills · proposals · trust_events · credentials · approval_challenges · ledger_entries · audit_events

দুটি স্পষ্ট সীমানা

1. **AI → Guardian:** AI-এর লেখা শুধু অবিশ্বস্ত প্রস্তাব। ব্যাংক অ্যাকাউন্ট, প্রাপক, অঙ্ক ও তারিখ সার্ভার যাচাই করে; AI সরাসরি executor ডাকতে পারে না।
2. **Guardian → Executor:** Guardian "অনুমোদনযোগ্য" বললেও লেনদেন *pending* থাকে। ব্যবহারকারীর ওই লেনদেনের জন্য নতুন WebAuthn assertion না এলে executor চলবে না।

অবস্থার ধাপ

DRAFT → CHECKED → PENDING_USER → APPROVED → SIMULATED_PAID
অথবা REJECTED / EXPIRED / CANCELLED

সার্ভারের state machine ছাড়া UI বোতাম দিয়ে অবস্থা পাল্টানো যাবে না। প্রতিটি transition-এর সময়, actor, কারণ ও পুরোনো/নতুন অবস্থা audit-এ লিখুন।

৪ · খরচ বুঝে পরামর্শ

ইনপুট

ব্যবহারকারী নিজের bank/MFS statement-এর CSV অথবা হাতে খরচ লিখবে। CSV-তে তারিখ, বিবরণ, অঙ্ক, debit/credit-এর কলামভেদ থাকতে পারে। আগে preview দেখিয়ে ব্যবহারকারীকে কলাম মেলাতে দিন; নাম/ফোন/অ্যাকাউন্ট নম্বর mask করুন। ব্যক্তিগত statement AI provider-এ পাঠানোর আগে স্পষ্ট সম্মতি নিন; সম্ভব হলে সারাংশই পাঠান।

হিসাবের ধাপ

1. CSV-এর encoding ও date format যাচাই; ভুল সারি আলাদা error report।
2. মুদ্রা **Decimal** বা integer poisha-তে রূপান্তর। float-এ টাকার হিসাব নয়।
3. কীওয়ার্ড নিয়মে খাবার, বিল, ভাড়া, যাতায়াত, subscription ইত্যাদি শ্রেণি; অনিশ্চিত হলে "review"। AI শ্রেণি প্রস্তাব দিতে পারে, চূড়ান্ত সত্য নয়।
4. একই প্রাপক + কাছাকাছি অঙ্ক + প্রায় মাসিক ব্যবধান দিয়ে পুনরাবৃত্ত খরচ শনাক্ত। ভুল subscription ব্যবহারকারী ঠিক করতে পারবে।
5. মাসিক/বার্ষিক মোট ও গত ৩ মাসের তুলনা Python-এ গণনা; AI শুধু যাচাইকৃত ফলকে সহজ বাংলায় ব্যাখ্যা করবে।

> **উদাহরণ:** "তিনটি streaming subscription-এ মাসে ৳৮০০, বছরে ৳৯,৬০০। কোনটি প্রয়োজন তা দেখুন।" সংখ্যা হিসাব থেকে এসেছে; AI বানায়নি।

এটি ব্যয় বোঝার সহায়তা; ব্যক্তিগত বিনিয়োগ বা লাইসেন্সপ্রাপ্ত আর্থিক পরামর্শ নয়। ডেমো ডেটা দিয়ে শুরু করুন, পরে সম্মতি নেওয়া বাস্তব CSV যোগ করুন।

৫ · বিল ও সময়সূচি

বিলের তথ্য: bill ID, অনুমোদিত প্রাপক ID, অঙ্ক, currency=BDT, শেষ তারিখ, invoice/reference, source, status। একই প্রাপক+reference+period আবার এলে duplicate হিসেবে থামান। ছবির OCR/AI তথ্য বের করলেও ব্যবহারকারী/নিয়ম দিয়ে যাচাই হবে।

সময় (উদাহরণ: ৭ অক্টোবর শেষ তারিখ)	কাজ
১০ দিন আগে	বিল শনাক্ত; প্রাপক, অঙ্ক, duplicate ও reserve যাচাই শুরু।
৭ দিন আগে	Guardian review; trust tier অনুসারে AI খসড়া নিজে প্রস্তুত করতে পারে।
৩ দিন আগে	অনুমোদন না থাকলে ব্যবহারকারীকে অনুরোধ ও reminder।
৫ অক্টোবর: safe date	অনুমোদন থাকলে simulated execution; না থাকলে স্পষ্ট "approval needed" সতর্কতা।
৭ অক্টোবর	বাকি থাকলে জরুরি সতর্কতা; কোনো নীরব auto-pay নয়।

সূত্র: safe_date = due_date − buffer_days; buffer ব্যবহারকারী ঠিক করবেন। Scheduler অ্যাপ বন্ধ থাকলেও চলার জন্য আলাদা scheduled worker/cron-এ রাখুন। একই bill বারবার চালালেও একটিই proposal/receipt হবে।

> **গুরুত্বপূর্ণ:** পূর্বে biometrics দিয়ে "সব ভবিষ্যৎ বিল" অনুমোদন নয়। প্রতিটি বিল/লেনদেনের নির্দিষ্ট প্রাপক, অঙ্ক ও reference দেখিয়ে নতুন WebAuthn অনুমোদন লাগবে। ব্যবহারকারী সাড়া না দিলে pending থাকবে, টাকা যাবে না।

৬ · Earned Autonomy ও Wilson lower bound

প্রতিটি কাজের ধরনের আলাদা বিশ্বাস রাখুন: নিয়মিত বিদ্যুৎ বিল, subscription, নতুন প্রাপক, বড় অঙ্ক। Shadow mode-এ AI "আমি এই খসড়া করতাম" বলে; ব্যবহারকারীর সিদ্ধান্তের সঙ্গে ফল মিলিয়ে গণনা হয়। প্রস্তাবের পর পাওয়া সত্যিকারের সিদ্ধান্তই live evidence।

$$\hat{p} = s/n;\ L = [\hat{p} + z^2/(2n) - z\sqrt{(\hat{p}(1-\hat{p})/n + z^2/(4n^2))}] / [1 + z^2/n],\ z = 1.96$$

এখানে n = যাচাইকৃত সিদ্ধান্তের সংখ্যা, s = নিরাপদ/সঠিক সিদ্ধান্তের সংখ্যা। 95% Wilson lower bound **L** ছোট নমুনায় অতিরিক্ত আত্মবিশ্বাস কমায়। n=0 হলে L=0।

Tier	উদাহরণ নীতি	কী স্বাধীনতা
0	শুরু / unsafe error	শুধু পরামর্শ; ব্যবহারকারী হাতে খসড়া নিশ্চিত।
1	n≥20, L≥0.80	চেনা বিলের খসড়া ও reminder নিজে বানায়।
2	n≥40, L≥0.90	চেনা বিলের safe date-এ pending approval queue বানায়।
3	n≥80, L≥0.95	নিয়মিত bill batch প্রস্তুত/অগ্রাধিকার দেয়, anomaly আগে থামায়।

২০/২০ হলে L≈0.839, ৪০/৪০ হলে ≈0.912, ৮০/৮০ হলে ≈0.954; তাই মূল গাইডের milestone বজায় থাকে। Threshold প্রকল্পের নীতি, প্রতিযোগিতার শর্ত নয়।

> **Unsafe ভুল:** AI যে লেনদেন করা উচিত ছিল না সেটি করার প্রস্তাব দিলে ধরনটির tier 0, pending proposal বাতিল, audit ও review। পুরোনো ৬–১২ মাসের replay ডেমোর evidence হতে পারে; outcome যাচাই, data leakage এড়ানো ও live evidence আলাদা দেখান। Replay একা production trust প্রমাণ করে না।

৭ · নিয়ম ও অনুমতি

Guardian AI নয়: পরীক্ষা-যোগ্য সাধারণ সার্ভার কোড। প্রত্যেক check `(pass, reason)` ফেরত দেয়; যে কোনো ব্যর্থতায় proposal থামে। AI লেখা পরিবর্তন করে নিয়ম শিথিল করতে পারবে না।

Check	ন্যূনতম আচরণ
প্রাপক	allowlist ID ও প্রদর্শিত নাম মেলে; নতুন/বদলানো প্রাপক user review।
অঙ্ক	positive integer poisha; ঐ প্রাপকের ছয় মাসের baseline-এর 2× ছাড়ালে hold; global hard cap।
বিল	reference+period duplicate নয়; due date যুক্তিযুক্ত; উৎস নথিভুক্ত।
তহবিল	demo balance ≥ amount এবং জরুরি reserve অবশিষ্ট।
বিশ্বাস	ধরন ও tier কেবল প্রস্তুতির অধিকার দেয়; final approval সব tier-এ দরকার।
সময়	expired/overdue হলে auto-preparation থামে; ব্যবহারকারীকে জানায়।
Injection	বিলের "সব নিয়ম উপেক্ষা কর" লেখা data হিসেবে থাকে; policy নয়। সন্দেহে hold + audit।

> **ভালো নকশা:** Guardian শুধু structured, server-validated bill record পড়ে। AI response বা OCR-এর raw instruction executor-এর কাছে যায় না। একই check approval-এর আগে এবং execution-এর ঠিক আগে আবার চালান।

৮ · প্রতিটি পেমেন্টে WebAuthn

1. ব্যবহারকারীকে **প্রাপক, ৳ অঙ্ক, বিল reference, fee=৳০ (ডেমো), due date** দেখান। "Approve this simulated payment" বোতাম চাপার পরেই WebAuthn শুরু।
2. সার্ভার cryptographically random, একবার ব্যবহারযোগ্য challenge দেয়। সার্ভারে তার সঙ্গে user ID, proposal ID, **canonical transaction hash**, expiry এবং status সংরক্ষণ করে।
3. ব্রাউজার `navigator.credentials.get()` কল করে `userVerification: "required"` সহ platform passkey ব্যবহার করে। ফোন/ল্যাপটপ স্থানীয়ভাবে fingerprint বা device PIN চাইতে পারে।
4. সার্ভার signature, challenge, origin, RP ID, credential ownership, UV flag, expiry ও replay যাচাই করে। এরপর DB transaction-এ proposal-এর বর্তমান তথ্যের hash আবার মেলায় এবং challenge consume করে।
5. Guardian পুনরায় পাস হলে একবারই simulated ledger লেখে। ব্যর্থ/বাতিল/timeout-এ কিছুই debit হয় না।

> **বায়োমেট্রিক তথ্য:** ওয়েব অ্যাপ fingerprint ছবি বা template নেয়/রাখে না; authenticator স্থানীয় verification করে, অ্যাপ public key ও signed assertion দেখে [২]। PIN fallback-কে একই user verification flow হিসেবে নথিভুক্ত করুন।

সাধারণ WebAuthn assertion নিজে payment details বিশ্বস্ত পর্দায় দেখায় না। অ্যাপ UI-তে বিস্তারিত দেখানো ও সার্ভারে challenge→transaction binding দরকার। হাই-রিস্ক বাস্তব পেমেন্টে PSP/bank-এর নিজস্ব authorization নীতি ও stronger trusted-display ব্যবস্থা আলাদা মূল্যায়ন করুন। HTTPS (localhost উন্নয়নে ব্যতিক্রম), স্থির RP ID ও origin কনফিগার করুন।

৯ · সিমুলেটেড লেজার ও audit

একবারই debit

সব টাকা integer poisha। প্রতিটি payment-এর একটি unique idempotency key = proposal ID। একই অনুরোধ দুবার এলে আগের receipt ফেরত দিন। DB transaction-এর ভেতর balance check, debit ও matching credit, state update, approval consume, audit write একসঙ্গে করুন; মাঝখানে failure হলে সব rollback।

Wallet liability −৳১,২০০ ↔ Demo merchant liability +৳১,২০০
receipt: SIM-2026-00042 · status: simulated_paid

এটি হিসাবের ডেমো, ব্যাংক settlement নয়। আলাদা demo fund top-up actor ছাড়া balance বদলানো যাবে না। কোনো negative balance নয়। প্রতিদিন ledger sum=0 ও wallet balance reconciliation চালান।

Audit event-এ রাখুন

timestamp (UTC), actor, action, proposal ID, bill ID, version, policy result ও reasons, trust tier/L, challenge ID (secret নয়), credential ID-এর নিরাপদ reference, receipt ID, prev_hash, event_hash। Hash chain স্থানীয় পরিবর্তন ধরতে সহায়ক; **এটি blockchain বা অপরিবর্তনীয় প্রমাণ নয়**। ব্যাকআপ/অ্যাক্সেস নিয়ন্ত্রণ প্রয়োজন।

> **প্রতিযোগিতার evidence:** SIM receipt, guardian log ও test run বাস্তব সিমুলেশন প্রমাণ করে। এগুলোকে on-chain transaction hash বলে জমা দেবেন না।

১০ · ফোল্ডার, সরঞ্জাম ও প্রথম সেটআপ

```
earned_autonomy/
  backend/
    main.py              # FastAPI routes
    models.py            # typed input/output
    ledger_db.py         # SQLite schema + atomic ledger
    advisor.py           # Decimal-based spending analysis
    bills.py             # due date + scheduler
    autonomy.py          # Wilson score + per-category tier
    checks.py            # deterministic Guardian rules
    agent.py             # optional AI extraction/explanation
    webauthn_service.py  # registration + transaction approval
    audit.py             # append-only audit events
    worker.py            # scheduled reminders / proposals
  frontend/              # React + Vite WebAuthn UI
  tests/                 # policy, flow, replay, ledger tests
  demo_data/             # synthetic CSV and bills
  .env.example  .gitignore  README.md
```

মূল গাইড থেকে রূপান্তর: `chain.py`, MetaMask, Sepolia, RPC, private key ও `web3.py` বাদ। `ledger_db.py` এখন payment simulation চালায়। আগের `autonomy.py`/`sim.py` থাকলে বাস্তবে খুলে পরীক্ষা করে পুনর্ব্যবহার করুন; ফাইল আছে ধরে নেবেন না।

1. Python 3.10+, Node.js, Git, VS Code; নতুন repo তৈরি।
2. Python virtual environment; FastAPI, Uvicorn, pytest, pandas, WebAuthn server library install করুন। লাইব্রেরির বর্তমান API অনুযায়ী উদাহরণ মিলিয়ে নিন।
3. Vite React app; browser WebAuthn API দিয়ে কাজ। `.env` Git-এ নয়; demo signing key নয়, শুধু optional AI API key।

১১ · ধাপে ধাপে: তথ্য থেকে খসড়া

ধাপ ১ · Database: transactions, bills, proposals, trust_events, credentials, approval_challenges, ledger_entries, audit_events তৈরি। foreign key, unique bill reference/period ও idempotency constraint দিন। synthetic seed data রাখুন।

ধাপ ২ · Advisor: CSV preview/column mapping, error row report, Decimal-এ মাসিক মোট, category ও recurring detector। সংখ্যার প্রত্যাশিত ফল হাতে যাচাই। AI থাকলে শুধু ভাষা/অনিশ্চিত শ্রেণি প্রস্তাব।

ধাপ ৩ · Bills: manual entry দিয়ে শুরু; পরে OCR/AI extraction। safe date গণনা, duplicate check ও reminder schedule। worker পুনরায় চালালে একই বিলের দ্বিতীয় proposal নয়।

ধাপ ৪ · Trust: shadow decisions জমা, category-wise Wilson score, tier update। replay আলাদা badge ও source ID সহ। unsafe approval suggestion-এ তাৎক্ষণিক tier 0।

ধাপ ৫ · Guardian: পর্ব ৭-এর checks ছোট function-এ লিখুন। raw bill text নয়, validated fields input। policy version রেকর্ড।

ধাপ ৬ · AI adapter: structured JSON schema; অজানা field reject। prompt injection test করুন। AI unavailable থাকলে manual data ও deterministic Guardian দিয়ে demo চলবে।

> **শুরু করার সহজ পথ:** প্রথমে ১টি প্রাপক, ১টি ৳১,২০০ বিল, ৳৫,০০০ demo balance এবং manual approval end-to-end চালান; তারপর AI ও CSV যোগ করুন।

১২ · approval, UI ও deployment

ধাপ ৭ · Passkey registration: লগইন করা ব্যবহারকারী server challenge নিয়ে platform credential নিবন্ধন করে। সার্ভারে credential ID/public key/user association রাখুন। জাল registration challenge ও ভুল origin reject করুন।

ধাপ ৮ · Payment approval: pending proposal খুলে বিস্তারিত দেখান; server challenge→transaction hash bind; browser assertion; সার্ভারে সব verification; atomic simulated execution। ব্যবহারকারী cancel করলে pending বা cancelled, debit নয়।

ধাপ ৯ · ওয়েবসাইট: ৫টি প্রধান view - (১) খরচ ও Advisor, (২) বিল ও safe date, (৩) category-wise trust ladder, (৪) approval inbox, (৫) history/audit। প্রতিটি পৃষ্ঠায় "DEMO · no real money" স্পষ্ট রাখুন।

ধাপ ১০ · Deploy: frontend ও backend HTTPS origin-এ; WebAuthn RP ID, allowed origin, session/cookie, CORS, CSRF এবং DB persistence কনফিগার। SQLite single-instance demo-তে ঠিক; বহু instance হলে transactional DB ব্যবহার করুন। scheduled worker আলাদা প্রক্রিয়ায়।

ডেমোর পাঁচ দৃশ্য

CSV → Advisor ফল; নতুন বিল → safe date; shadow/replay → tier; approval inbox → fingerprint/PIN prompt → SIM receipt; বদলানো প্রাপক বা injection → Guardian hold + audit। ভিডিওতে ডেমো সীমা ও track অবস্থান বলুন।

১৩ · লেনদেন সম্পাদনের pseudocode

```
def create_approval_challenge(user, proposal_id):
    p = db.get_pending_owned_proposal(user, proposal_id)
    require(guardian.check(p).ok)
    tx = canonical_json({"payee_id": p.payee_id,
       "amount_poisha": p.amount_poisha,
       "currency": "BDT", "bill_ref": p.bill_ref,
       "proposal_id": p.id, "version": p.version})
    challenge = secure_random_32_bytes()
    db.store_challenge(user, p.id, sha256(tx),
                       challenge, expires_in=120, used=False)
    return webauthn_options(challenge,
              userVerification="required")

def approve_and_simulate(user, proposal_id, assertion):
    with db.atomic_transaction():
        p = db.lock_pending_owned_proposal(user, proposal_id)
        c = db.lock_unused_challenge(user, p.id)
        require(not c.expired)
        require(sha256(canonical_payment(p)) == c.tx_hash)
        webauthn.verify(assertion, challenge=c.challenge,
            expected_origin=ORIGIN, rp_id=RP_ID,
            stored_public_key=user.credential_public_key,
            require_user_verification=True)
        require(guardian.check(p).ok)
        require(db.demo_balance(user) >= p.amount_poisha + RESERVE)
        db.consume_challenge(c)
        receipt = db.post_balanced_entries_once(
            idempotency_key=p.id, amount=p.amount_poisha)
        db.mark_simulated_paid(p, receipt)
        db.append_audit(user, p, receipt)
    return receipt
```

এটি নকশার pseudocode, নির্দিষ্ট library-এর runnable API নয়। challenge random এবং সার্ভারে *একটি নির্দিষ্ট অপরিবর্তিত transaction hash*-এর সঙ্গে বাঁধা। অনুমোদনের পরে প্রাপক/অঙ্ক বদলালে hash মিলবে না। নতুন অনুমোদন চাইতে হবে।

১৪ · Threat model ও সৎ সীমাবদ্ধতা

ঝুঁকি	প্রতিরোধ / বাকি সীমা
ভুয়া বিল বা prompt injection	OCR/AI output data; allowlist, amount, duplicate, human review; সন্দেহে hold ও tier 0। নিখুঁত শনাক্তকরণ দাবি নয়।
Approval-এর পর payee/amount বদল	canonical hash→challenge binding, version lock, execution-এর আগে recheck।
Replay / double click / worker race	single-use expiring challenge, unique idempotency key, DB lock ও atomic transaction।
চুরি হওয়া session / device	HTTPS, secure cookie, session timeout, user verification; device PIN জানা অন্য ব্যক্তি ঝুঁকি।
AI ভুল অঙ্ক বা ভুল উপদেশ	Decimal/server হিসাব, schema validation, review; AI শুধু ব্যাখ্যা।
Server/DB compromise	least privilege, backups, audit hash chain; local audit নিজে অপরিবর্তনীয় নয়।
বাস্তব PSP API অপব্যবহার	prototype-এ কোনো live API/credential নয়। পরবর্তী integration-এ provider-এর লিখিত অনুমোদন, চুক্তি, security review ও প্রযোজ্য Bangladesh Bank শর্ত দরকার [৩]।

বাংলাদেশে blockchain “কাজ করে না” এমন সার্বিক দাবি করা ঠিক নয়। এই প্রকল্পের অন-চেইন অংশ বাদ দেওয়ার কারণ: বাংলাদেশি বিল payment flow-এর সঙ্গে প্রাসঙ্গিকতা ও প্রতিযোগিতার নির্দিষ্ট track-এ fit।

১৫ · ন্যূনতম ২০টি acceptance test

#	পরীক্ষা	প্রত্যাশিত ফল
01	সঠিক CSV আমদানি	মোট ও category সঠিক
02	ভুল date/amount সারি	error report; অর্থ লেজারে নয়
03	পুনরাবৃত্ত subscription	মাসিক/বার্ষিক মোট সঠিক
04	AI বানানো অঙ্ক	server হিসাবই দেখায়
05	safe date ও buffer	সঠিক দিন গণনা
06	duplicate bill reference	দ্বিতীয় proposal নয়
07	অজানা প্রাপক	Guardian hold
08	অস্বাভাবিক 2× অঙ্ক	Guardian hold
09	অপর্যাপ্ত demo balance	debit নয়
10	জরুরি reserve ভাঙে	debit নয়
11	বিলের injection text	নিয়ম অপরিবর্তিত; audit
12	tier 0-তে auto prepare	নিষিদ্ধ
13	২০/২০, ৪০/৪০, ৮০/৮০	Wilson tier 1/2/3
14	unsafe AI suggestion	tier 0; pending বাতিল
15	high tier, assertion নেই	কোনো execution নয়
16	ভুল origin/RP ID বা UV=0	approval reject
17	expired/replayed challenge	reject; debit নয়
18	approval-এর পর অঙ্ক বদল	hash mismatch; reject
19	দুইবার click/worker race	একটি balanced receipt
20	overdue বা user cancel	alert; debit নয়

একই adversarial test একটি "শুধু LLM" baseline-এ চালান। কতটি unsafe proposal/approval হয়েছে, প্রকৃত সংখ্যা ও পদ্ধতি লিখুন; অনুমান নয়।

১৬ · সময়রেখা, জমা ও প্রমাণ

দিন	অগ্রাধিকার
১৮ অক্টোবর	দল/track ঠিক; FastAPI+React skeleton; SQLite ledger; একটি bill → Guardian → pending। আয়োজককে Topic A/C/F fit ও বিস্তারিত requirement জিজ্ঞাসা।
১৯ অক্টোবর	Advisor, scheduler, Wilson trust, WebAuthn registration/approval, ২০ acceptance test।
২০ অক্টোবর	HTTPS demo deployment, end-to-end rehearsal, audit, ভিডিও ও deck; দাপ্তরিক জমার নির্দেশে নির্দিষ্ট cutoff যাচাই করে আগেই জমা।

জমার evidence pack

- ইংরেজি ১ পৃষ্ঠার problem + scope: simulated BDT payments; no blockchain; selected topic-এর কারণ।
- Architecture/flow, Guardian policy, Wilson formula ও trust evaluation table।
- GitHub repo, README, synthetic demo data, deploy/run guide; গোপন key ছাড়া।
- ২০ test-এর pass/fail report, baseline comparison, নিরাপত্তা self-assessment।
- ≤৫ মিনিট ডেমো ভিডিও ও ≈১০ মিনিট ইংরেজি deck (মূল গাইডের প্রস্তুতি লক্ষ্য; দাপ্তরিক জমার ফাইল সীমা যাচাই করুন)।
- SIM receipt, audit trail, failed injection ও tamper test-এর screenshot। এগুলো on-chain proof নয়।

> **শেষ checklist:** □ প্রত্যেক final payment-এ নতুন user verification □ payee/amount bound challenge □ no biometric capture □ no live PSP call □ audit/receipt সঠিক লেবেল □ topic claim সৎ □ দল ও deadline যাচাই।

১৭ · যাচাইকৃত উৎস ও এখন কী করবেন

উৎস

1. **প্রতিযোগিতার আন্তর্জাতিক ট্র্যাক:** IEEE CIS Singapore, "2026 FinTechathon" - Topic A/C/F, দল ও সময়সূচি।
   https://www.ieee-cis-singapore.org/2026-fintechathon

2. **WebAuthn Level 3:** W3C - challenge, user verification, local biometric/PIN ও UV flag।
   https://www.w3.org/TR/webauthn-3/

3. **বাংলাদেশের PSP/PSO অনুমোদন:** Bangladesh Bank, "Approval Procedure of Payment System Operator / Payment Service Provider", এবং Payment Systems তথ্য।
   https://www.bb.org.bd/aboutus/regulationguideline/psd/pso_psp_03022019.pdf
   https://www.bb.org.bd/en/index.php/financialactivity/paysystems

উৎসগুলি ১৮ অক্টোবর ২০২৬-এ দেখা। প্রকাশিত সংক্ষিপ্ত প্রতিযোগিতা বিবরণ থেকে "real on-chain transaction hashes" বাক্যটির বিস্তারিত অফিসিয়াল requirement স্বাধীনভাবে নিশ্চিত করা যায়নি; মূল গাইডে এটি লেখা আছে। তবে Topic F যে on-chain, তা প্রকাশিত বিবরণে স্পষ্ট। কোনো দাবি জমা দেওয়ার আগে আয়োজকের মূল নথি/উত্তর মিলিয়ে নিন।

এখনই প্রথম তিন কাজ

1. দল ও জমার বিষয় ঠিক করুন; সাধারণ বাংলা বিল প্রকল্প হলে Topic A-র সঙ্গে fit পরীক্ষা করুন।
2. একটি synthetic bill ও ৳৫,০০০ demo balance দিয়ে Guardian→pending→WebAuthn→SIM receipt চালান।
3. ২০ test-এর expected result আগে লিখে রাখুন; সফলতার সংখ্যা ও ব্যর্থতা সৎভাবে রিপোর্ট করুন।

> এই নথি নির্মাণ নির্দেশিকা। বাস্তব আর্থিক সেবা চালুর অনুমতি বা নিরাপত্তা সনদ নয়।